

Document Number: P1080R0

Date: 2018-05-06

Audience: SG16

Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [March 28th, 2018](#)
- [April 11th, 2018](#)
- [April 25th, 2018](#)

March 28th, 2018

Draft agenda:

- Jacksonville recap.
- Divide and conquer: can we focus efforts on different support areas?
 - Views, ranges, code units vs code points vs EGCs.
 - Encoding, decoding, normalization, segmentation.
 - UCD and CLDR interfaces.
 - Localization, collation, case mapping.

Attendees:

- Corentin Jabot
- Florin Trofin
- JeanHeyd Meneide
- Mark Zeren
- Martinho Fernandes
- Michael Spencer
- Nicole Mazzuca
- Peter Bindels
- Tom Honermann
- Zach Laine

Meeting summary:

- This was our first official teleconference meeting as SG16. We discussed and agreed on a number of steps to take as part of the transition from the informal std-text-wg group:
 - The recently added std-text-wg@googlegroups.com mailing list will be removed in favor of an isocpp sponsored mailing list on open-std.org once it is created.
 - Tom confirmed with Vinnie Falco that the C++ Alliance is looking into acquiring a paid Slack plan. We'll continue using Slack for now, but will rename the existing "std-text-wg" channel to "sg16-

- unicode" or, if we can't rename it, migrate to a new channel with the new name.
 - We'll rename the existing std-text-wg github repo to "sg16-unicode". After the meeting, JeanHeyd suggested that we should migrate to a repo not tied to a personal account and Tom agreed to do so.
- We then went on to discuss future plans.
- Mark suggested that we plan to get together with Apple Unicode developers before, during, or after the San Diego meeting. Mark previously worked with a number of Apple developers that work on Unicode and they have considerable expertise that we would love to have available.
- We then discussed ideas we could pursue for C++20:
 - Mark expressed interest in cleanup of the basic_string specification. For example:
 - Fix long standing issues that were not fixed in C++11.
 - Cleanup iterator invalidation rules to match existing implementations.
 - Mark also expressed interest in adding an uninitialized append capability to vector and basic_string.
 - Mark suggested that we start researching whether and how the ISO standard can reference the Unicode standard. Questions include:
 - How do we handle the different cadence of ISO C++ releases and Unicode releases?
 - Do we allow implementors to choose a Unicode version?
 - Would it suffice to reference other ISO/IEC standards such as ISO/IEC 10646 and ISO/IEC 14651 that reflect portions of the Unicode standard?
 - <https://www.iso.org/standard/56921.html>
 - <https://www.iso.org/standard/68309.html>
 - Tom remarked that ISO C and C++ do not specify the encoding of u"" and U"" string literals, but suspects that all current compilers encode them as UTF-16 and UTF-32 respectively. If this can be confirmed, then we could propose mandating UTF-16 and UTF-32 respectively to WG14 and WG21.
 - Tom expressed a desire to modernize the terminology used in the C++ standard. In particular, replacing or clarifying uses of "character" and "character set" with more modern terms like "code unit", "code point" and "character encoding". This will be necessary for future specification and gives us an opportunity to educate the committee on some of these distinctions.
 - Tom also suggested that we could review the standard for interfaces that are too broken to be fixed and deprecate them. For example, `std::ctype::toupper()`.
 - Nicole suggested that, for `std::ctype::toupper()` in particular, we could propose new interfaces that provide the same behavior, but under names that make the limitations clear. For example, `ascii_toupper()` or `basic_toupper()`.
 - Tom discussed the difficulties we face today in drafting wording updates and that, ideally, we would have some kind of tool that would allow us to edit a fork of the standard LaTeX sources, and then build it such that only the sections that were modified would be present in the resulting document (with appropriate insert/delete markup). In discussion with Richard Smith in Jacksonville, Richard noted that he has wanted something like that for some time. A considerable benefit of such an approach is that it would make the process of updating wording for more recent drafts much simpler.
- We next discussed some goals for Rapperswil:
 - Tom will pursue getting char8_t through CWG and LWG; likely via a delegate.
 - Topics for SG16 to discuss in session:
 - What is our long term vision?
 - What might a TS contain?
 - Work group activities:
 - Identify use cases. We've been talking about getting a solid list of common text manipulation use cases together for approximately forever now. Perhaps we could devote some time to doing that work?
- Finally, Peter suggested that we could propose a library-in-a-week project for the upcoming C++Now conference, May 6th-11th.
- Tom noted that many of us have our own pet projects at the moment, many of which overlap considerably. Tom has `text_view`, Zach has `text`, Martinho has `Ogonek`, JeanHeyd has his own `text_view`, Corentin has his own experiment with normalization, etc... How can we best collaborate and execute on a shared vision for the standard?

April 11th, 2018

Draft agenda:

- Identify champions to research and author papers for projects discussed in the last meeting:
 - Terminology modernization
 - Mandate char16_t and char32_t string literals be UTF-16/UTF-32.
 - Identify existing features to deprecate/replace.
 - basic_string specification cleanup.
 - uninitialized append for contiguous containers.
- Determine how to contribute to and collaborate on a common code repository; building from the ground up.

Attendees:

- Florin Trofin
- JeanHeyd Meneide
- Mark Zeren
- Nicole Mazzuca
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- We first discussed some administrative updates:
 - We now have a mailing list at <http://www.open-std.org/mailman/listinfo/unicode>! Thanks to JeanHeyd for an initial post summarizing our current projects and goals.
 - Our Slack channel has been renamed to reflect our new SG16 branding. Thanks to Mark for taking care of that.
 - We have a new SG16 GitHub organization at <https://github.com/sg16-unicode>. The old std-text-wg repo will be retired (but preserved). Meeting summaries will now be tracked in a new sg16-meetings repository. We'll create additional repositories as needed.
- We then spent a little time discussing how best to make use of the new GitHub org.
 - Tom created GitHub projects for the initiatives discussed in our last meeting and asked for feedback regarding tracking things that way. None of us have much familiarity with GitHub projects, so there wasn't much feedback. Some experimenting will be required. Tom noted that task ownership seems to depend on creating issues associated with a git repo.
 - Nicole noted that, because we now have a central org, we can create numerous repos.
 - Steve mentioned that having a repository just for papers is very useful.
 - Tom suggested creating a "sg16" repo with a Readme.md that provides an overview of SG16 and other introductory text; for example, links to the mailing list and Slack. Basically, a repo to host the Readme.md file from the old std-text-wg repo. We might also use this repo for general management purposes; e.g., to host GitHub issues used for project tracking.
 - Zach asked about where to host our mythical use cases doc and suggested an approach in which the use cases are stored as code rather than as documentation. This could support testing the use cases with various implementations (for correctness, performance, expressiveness, etc...). Should we decide to submit a use cases paper, the paper could presumably pull directly from the code.
 - It was mentioned that being able to share code samples on godbolt.org (or another Compiler Explorer host) would be helpful. Nicole suggested we reach out to Matt Godbolt to see if he would

be amenable to adding Unicode related projects such as ICU. Matt has fulfilled similar requests in the past; e.g., for range-v3.

- We then moved on to identifying champions to research and author papers for projects discussed in the last meeting.
 - Steve volunteered to work on updating normative references to ISO/IEC 10646 to a revision that isn't older than some committee members.
 - Tom and Zack agreed to work on terminology modernization.
 - Martinho wasn't present for the meeting, but had previously indicated interest in working on a paper to mandate that `char16_t` and `char32_t` string literals be UTF-16/UTF-32; he reconfirmed this on Slack after the meeting. Tom volunteered to help try and identify any existing compilers that don't use UTF-16/UTF-32.
 - Mark reaffirmed his intent to work on `basic_string` specification cleanup.
 - Mark also reaffirmed his intent to work on uninitialized append for contiguous containers.
 - Noone volunteered to champion work on identifying existing features to deprecate/replace.
- We briefly discussed deprecation approaches using `std::toupper()` as an example. It was acknowledged that eventual removal of this function may not be feasible due to widespread use. Likewise, automatic refactoring may not be feasible since a suitable replacement function might require multiple code points (either for context or for mapping). Nicole noted that this is an example of a function that can remain useful, but has a name that does not communicate its limitations. Deprecating it in favor of an appropriately named alternative (e.g., `ascii_toupper()`) would support automatic refactoring.
- Our next discussion focused on further collaboration.
 - Tom again expressed a desire to bring our collective efforts together to enable collaborating on a reference implementation of features we'd like to see in a future TS. Perhaps taking a bottom up approach to building it.
 - Zach noted that his `text` implementation and documentation are designed around three distinct layers: strings, Unicode, and text.
 - JeanHeyd expressed an interest in surveying `Ogonek`, `text`, `text_view`, etc... to identify overlap and further define layering opportunities.
- Tom then asked about recent reflector discussions regarding `string_view` and what we might learn from them that would be applicable to view/reference types we might design.
 - Zach noted a few guidelines:
 - Don't support default comparisons between different kinds of views (e.g., `string_view` and `rope_view`).
 - Don't provide operators that hide complexity. For example, an operator+ for text/string views that would require allocation might be surprising.
 - Owning vs non-owning is more important than shallow vs deep compare.
 - Mark mentioned that `string_view` cannot support intrinsic optionality because a null pointer state is observable (via `data()`).
- We briefly discussed $O(1)$ complexity requirements on `begin()`.
 - Tom mentioned that his `itext_iterator` types do not currently meet this requirement because, given an ill-formed initial code unit sequence, `begin()` may consume an unbounded number of code units. The consumption is necessary to ensure that, in the case where the end of the underlying code unit sequence is reached before any code points are successfully decoded, that `begin() == end()` will hold.
 - Zach noted that his iterators don't encounter this situation because the `text` type ensures that the underlying code unit sequence is always valid. His transcoding iterators also do not hit this because they produce a replacement character for each ill-formed sequence (presumably limited to a bound length). This would be an issue for support of an error policy that simply skips ill-formed code unit sequences though.
- Florin requested that we make sure to keep IBM users in mind and that we work with people from IBM to ensure that what we propose won't be problematic for non-ASCII based systems. Tom fervently agreed and indicated he has maintained contact with Hubert Tong.
- We finished by agreeing to two meetings to be held before the pre-meeting mailing deadline for Rapperswil. The deadline is May 7th; we will meet ~April 18th and May 2nd~ [Post-meeting writeup]

correction: we will meet just once on April 25th].

April 25th, 2018

Draft agenda:

- Review and discuss any draft papers targeting the Rapperswil pre-meeting mailing.
- Discuss plans and goals for those attending Rapperswil.

Attendees:

- Bob Steagall
- Dalton Woodard
- JeanHeyd Meneide
- Martinho Fernandes
- Peter Bindels
- Sergey Zubkov
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- We started off with introductions from first time attendees Dalton and Sergey.
- Tom provided a few administrative updates:
 - A new sg16 repo was created under the sg16-unicode github org; the old std-text-wg repository is now retired.
 - <https://github.com/sg16-unicode/sg16>
 - A new sg16-papers repo was not created under the assumption that it would be simpler to just use the new sg16 repository.
 - The old std-text-wg mailing list was removed; old emails were forwarded to the new mailing list.
 - The github projects that Tom was experimenting with have been removed in favor of tracking via github issues on the sg16 repository.
 - Alisdair provided some char8_t wording review; Mark is off the hook for doing so.
- We then did a round of status updates.
 - Steve provided a brief status update for the project to update normative references to ISO/IEC 10646
 - <https://github.com/sg16-unicode/sg16/issues/1>
 - Further discussion was postponed until after status updates.
 - Martinho provided a brief status update for the project to mandate that char16_t and char32_t literals use UTF-16 and UTF-32 encodings respectively.
 - <https://github.com/sg16-unicode/sg16/issues/6>
 - Further discussion was postponed until after status updates.
 - Zach provided an update on Boost text and reported having fun adding support for the Unicode bidirectional algorithm.
 - Jeanheyd reported that he is working on benchmarks and comparisons of Ogonex, Boost text, text_view, ICU, and other libraries.
 - We briefly discussed some of the work that Bob Steagal has been doing. Bob had previously shared some UTF-8 conversion performance numbers with Zach and Tom. Bob had not yet joined the meeting, so Zach gave a brief overview. When Bob later joined in, we discussed further (see below).
- We then returned to discussion on normative updates for Unicode standard references:

- A productive discussion on Slack earlier in the day was helpful in setting direction. At issue was how to handle UCS-2 and UCS-4 references in the standard since definitions of those terms would be lost by updating the normative reference to a recent standard. It was confirmed that UCS-2 and UCS-4 are only referred to by the deprecated codecvt facets in annex D.
- Tom had suggested in the Slack discussion that we could remove the deprecated features in C++20. This would remove the existing references and avoid the need to retain any definitions for UCS-2 and UCS-4.
- Zach noted that the standard practice for deprecation is to deprecate first and replace/remove in a future standard.
- Steve noted that Debian code search reveals uses of the deprecated codecvt facets.
- Martinho suggested the deprecated facets could be specified to use UTF-16/UTF-32 instead of UCS-2/UCS-4. This would be a technical break, but perhaps not a significant concern.
- Tom noted that this would definitely break `codecvt_utf16` since it exists solely to convert between UTF-16 and UCS-2/UCS-4.
- Steve stated that we can retain the old ISO/IEC 10646 reference for the deprecated UCS-2 and UCS-4 references, and use an updated reference for everything else.
- Steve noted that LWG thought they had previously addressed the UCS-2/UCS-4 issue by deprecating existing uses, but since references still remain, the issue is not really addressed.
- Tom asked how we should move forward. With one paper addressing both the normative update and the UCS-2/UCS-4 references? Two papers? Perhaps three?
- Zach expressed a preference to address separate concerns separately.
- Tom expressed a preference for one paper with wording to address both issues. The intent being that, if that approach were to fail, we could fall back to separating out the issues. This preference is intended to avoid dependencies between the two issues; to avoid the case where we end up updating the normative reference, but not removing the UCS-2 and UCS-4 references, thus leaving undefined terms in the standard.
- Sergey asked about the possibility of updating the deprecated facets to specify that the encoding conversion is implementation defined.
- Tom: Someone (not sure who) had previously (not in this meeting) noted that there was already some implementation divergence regarding whether the codecvt facets actually do convert between UCS-2/UCS-4 vs UTF-16/UTF-32.
- Martinho noted that the difference is unlikely to matter in real world usage because lone surrogates are unlikely to be present.
- Peter countered that lone surrogates may appear in file names (on Windows where file names have 16-bit code units).
- Zach stated that Windows no longer allows file names that are ill-formed UTF-16.
- Tom noted he had heard this, but hasn't seen it confirmed.
- Peter noted that WTF-8 exists to handle UCS-2 and malformed UTF-16.
- Martinho stated that there is a difference between ill-formed UTF-16 and text that is not actually UTF-16.
- We confirmed Steve had the guidance he felt he needed from the group to proceed as he deemed fit.
- Steve stated that he will circulate papers when ready.
- We then returned to discussion on the character encoding for `char16_t` and `char32_t` literals.
 - Martinho have a quick overview of his draft:
 - https://github.com/rmartinho/sg16/blob/master/proposals/p0000r0_utf16_32_literals.md
 - Tom reported reaching out to the author of WG14 N2245 (<http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2245.htm>). That author confirmed no known C compiler implementations that don't use UTF-16/32.
 - Zach said the paper looks good and asked what happens with `\u` escapes in ordinary string literals today.
 - Tom stated that is implementation defined.
 - Zach and Martinho noted that there are potentially different results for escapes vs conversion from source encoding.

- Tom suggested updating the character literal wording to use consistent naming with the updated string literal wording.
 - Tom also noted there would be a merge conflict with the char8_t proposal.
 - Martinho noted that he had also observed that. He also observed that the term, "UTF-8 character literal" is defined but never referenced. Martinho confirmed he will define character/string literals using char16_t/char32_t and UTF-16/32 terms to match the UTF-8 related definitions.
- JeanHeyd asked about where to create new repositories for new projects.
 - Zach and Steve both suggested creating personal repositories; we can transfer ownership later if/when it becomes appropriate.
- Steve observed that it would be helpful to use a consistent license for the works we produce.
 - JeanHeyd asked what license is preferred, CC0, Boost, MIT?
 - Zach stated a preference for avoiding CC0 because of legal complexities with "public domain".
 - Tom expressed no particular preference but observed that CC0 is more complicated from a wording perspective.
 - Peter stated we should use a license that is friendly to implementors.
 - Zach responded that fear of patent bombing prevents usage in some cases.
 - We settled on using the Boost license for group projects.
- Bob then provided an introduction and shared more specifics of his work.
 - He has been working on UTF-8 conversion performance improvements and shared some results that he will be presenting at C++Now.
 - Peter asked if more benchmarks could be added to Bob's tests.
 - Bob answered yes and noted that Zach did the work to integrate Boost text.
- Steve reminded everyone to request paper numbers for the pre-Rapperswil meeting now.
- We confirmed that the next meeting will be held on May 16th; three weeks from now so as not to conflict with C++Now.